Citrate: Protocol Specification for an AI-Native BlockDAG Network

GhostDAG Consensus, the Lattice Virtual Machine, and the Model Context Protocol

Larry Klosowski Lauren Mendenhall
Citrate Inc.
Larry@citrate.ai

The Gradient Papers, No. I
Preprint — not yet peer reviewed

Maturity: [Verified]. The February 2026 draft of this paper declined to publish figures it could not measure. Most of what it labelled *Specified* or *Planned* is now live on the chain-id 40204 testnet. This revision re-anchors every mechanical claim to a file path or a deployed contract address, and it keeps the earlier draft's discipline: where a number is a documented target rather than a measured result, it is labelled as one.

0.1 Abstract

We present the protocol specification for the Citrate Network, a BlockDAG system that extends GhostDAG consensus with AI-native execution. The architecture comprises three layers: a **GhostDAG consensus layer** providing parallel block production with committee BFT finality checkpoints (k=18, up to 10 parents per block, a 100-validator committee with a 67-signature quorum); a **Lattice Virtual Machine (LVM)** providing EVM bytecode compatibility via REVM, augmented with families of AI-specific precompiled contracts for inference, verifiable proof checking, deterministic fixed-point tensor compute, and paraconsistent learning aggregation; and a **Model Context Protocol (MCP)** layer providing OpenAI- and Anthropic-compatible REST endpoints for model discovery, inference, and orchestration. This paper documents the implemented components, the consensus engine, the virtual machine with tested AI precompiles, dual ECDSA/Ed25519 identity, and SALT tokenomics realized in deployed contracts, and distinguishes them from components that remain feature-gated or in design. The consensus layer is integrated with the federated learning protocol described in companion Paper II, whose Belnap aggregation precompile is live. The native token SALT (1 billion supply) functions as gas, staking collateral, and governance weight. As of this revision the testnet carries **39 deployed contracts** on chain 40204 and the node's core workspace holds **3,517 tests**.

Keywords: BlockDAG, GhostDAG, BFT finality, EVM compatibility, AI precompiles, Model Context Protocol, verifiable inference, Citrate Network

0.2 1. Introduction

Citrate is a three-layer blockchain designed for AI-native applications. The thesis is that a general-purpose execution environment with on-chain AI primitives, model registration, verifiable inference,

deterministic tensor operations, adapter management, and model orchestration, can serve as infrastructure for decentralized AI in a way that purpose-built AI coordination networks (which lack general programmability) and general-purpose blockchains (which lack AI-specific optimizations) cannot.

The design draws on three established technologies. First, **GhostDAG consensus**, originated by Sompolinsky and Zohar [1, 2] and first productionized by Kaspa, provides a BlockDAG structure enabling parallel block production with deterministic ordering. Second, the **Ethereum Virtual Machine**, the most widely deployed smart-contract runtime, provides developer tooling, auditing infrastructure, and application compatibility. Third, the **Model Context Protocol**, originated by Anthropic [23], provides standardized interfaces for model-to-model and model-to-tool communication.

Since the February 2026 draft, a category of AI-native and inference-verifying blockchains has matured, and this specification positions Citrate against it explicitly (Section 7): the systematization of decentralized AI (DeAI) by Zhang et al. [31]; proof-of-logits and subnet-scored designs (Ambient, Bittensor's Yuma consensus) [13]; and optimistic and TEE-attested inference rollups [32]. Citrate's distinguishing choice is to embed AI operations as composable precompiles inside a general EVM, rather than as an isolated task market.

This paper is organized as a protocol specification. Section 2 describes the consensus layer; Section 3 the Lattice Virtual Machine and its AI precompiles; Section 4 the MCP orchestration layer; Section 5 SALT tokenomics; Section 6 security; Section 7 comparative position; Section 8 open problems.

Implementation status. Claims marked **[Implemented]** describe components built and tested on the chain-40204 testnet, with a file path or contract address given. Claims marked **[Specified]** are designed in detail but not yet built. Claims marked **[Feature-gated]** exist in code but are compiled out of the default build. This convention lets a reader distinguish what exists from what is proposed, and check each claim against the source.

0.3 2. Consensus Layer: GhostDAG with BFT Finality

0.3.1 2.1 GhostDAG Overview

[Implemented] Citrate's consensus layer implements the GhostDAG protocol [1, 2], which generalizes Nakamoto consensus to directed acyclic graphs (`core/consensus/src/ghostdag.rs`, `GhostDag` at line 54). Blocks reference multiple parents (up to 10 in Citrate: one selected parent plus up to nine merge parents; `core/consensus/src/types.rs:179`), forming a DAG rather than a chain. The protocol partitions blocks into a blue set (consistent with the k-cluster rule, anticone size $\leq$ k) and a red set. A selected-parent chain through the blue set provides deterministic total ordering for contract execution (`tip_selection.rs`, `ordering.rs`, `chain_selection.rs`), and blue scores serve as the primary tip-selection weight (`calculate_blue_set` at `ghostdag.rs:201`).

The **k parameter is 18** (`core/consensus/src/types.rs:178`), calibrated for the network's block time and expected propagation delay. For context, Kaspa's Crescendo hardfork (May 2025) raised k to 124 at 10 blocks per second with max parents 16 [30]. Citrate's more conservative k=18 reflects a slower block rate chosen to accommodate the larger blocks required by AI-augmented blocks carrying embedding data (Section 2.3). The k-cluster rule is exercised by roughly twenty consensus test files, including `k_cluster_tests.rs`, `fork_choice_tests.rs`, `h07_reachability.rs`, and `proptest_consensus.rs`.

0.3.2 2.2 Block Time and Rate

[**Implemented**] The chain-40204 testnet targets a **2-second block time** (`testnet-config.toml:4`; `core/sequencer/src/block_builder.rs:76`, `block_time_target: 2`); the devnet runs at 1 second. This is more conservative than the February 2026 draft, which described a 0.5-second target; the code and the running testnet are the ground truth, and this revision corrects the figure. The design rationale is unchanged: AI-augmented blocks are larger than pure-transaction blocks. When the federated learning path of Paper II is active, each block carries embedding vectors alongside transactions; a 768-dimensional float32 embedding adds roughly 3 KB per block, comfortably within the maximum block size at a 2-second cadence. Block time remains a governance-adjustable parameter, so the rate can be raised if embedding compression advances or the learning path is not needed on a given deployment.

0.3.3 2.3 BFT Finality Checkpoints

[**Implemented**] Citrate runs a dual finality mechanism. Depth-based optimistic confirmation is provided by the `FinalityTracker` (`core/consensus/src/finality.rs:95`). On top of it, **committee BFT checkpoints** (`core/consensus/src/checkpoint.rs`, "WP-S.3: Committee BFT Checkpoints") select a committee deterministically by VRF seed and validator public key, and require a **67-of-100 quorum** (2/3+1; `checkpoint.rs:88,103`) over a domain-separated ed25519 vote (`CITRATE-CHECKPOINT-V1`, `checkpoint.rs:59`). The default checkpoint interval is **50 blocks** (`checkpoint.rs:101`); when a checkpoint reaches quorum it overrides depth-based finality up to the checkpoint height, and the committed block hash, blue set, and state root become irreversible. The README documents a 12-second optimistic finality target; the checkpoint cadence itself is 50 blocks, and this revision states both rather than collapsing them into a single headline number. The checkpoint commitment is extensible to learning state (routing weights, adapter registries), which is how Paper II's learning checkpoints bind to finality (`core/learning/src/checkpoint.rs:14`).

Safety holds if fewer than n/3 committee members are Byzantine, by standard BFT arguments [11]; with a 100-validator committee requiring 67 signatures, this tolerates up to 33 Byzantine validators. Liveness holds under GhostDAG's blue-set property when fewer than n/2 of validators are Byzantine [1, 2].

0.3.4 2.4 Performance: What Is and Is Not Measured

The February 2026 draft declined to publish a throughput figure without a measurement methodology. That discipline is retained, and can now be stated precisely.

[**Implemented, measured**] The parallel executor is benchmarked with a gate that asserts an **8-worker throughput of at least 2× the 1-worker throughput** on disjoint-sender workloads (`core/execution/benches/tps_parallel.rs:10`). This relative speedup is the claim the in-repo benchmark actually proves. The executor's absolute ceiling on disjoint transfers is high (prior sprints measured hundreds of thousands of transfers per second at the executor layer), but the executor ceiling is not the network TPS.

[**Documented target, not benched**] The README advertises 5,000 TPS sustained with a 10,000 TPS ceiling (`README.md:65`). This is a documented target, not a value produced by the in-repo benchmarks, and the benchmark header itself disclaims its absolute throughput as executor-ceiling-bound (`tps_parallel.rs:145`). We report it as a target and label it as such. Real-world network throughput is bounded by signature recovery, RLP decode, mempool gossip, and disk I/O; those are the optimization targets for subsequent work.

0.4 3. Execution Layer: The Lattice Virtual Machine

0.4.1 3.1 EVM Compatibility

[Implemented] The Lattice Virtual Machine (LVM) executes EVM bytecode via REVM (`core/execution/src/r` registering Citrate's custom precompiles through `register_citrate_precompiles`. Solidity, Vyper, or Yul compiled for Ethereum deploys on Citrate without recompilation, giving immediate access to Hardhat, Foundry, OpenZeppelin, Ethers.js, and existing audit tooling. Parallel execution with MVCC state is implemented in `core/execution/src/parallel/` and `core/execution/src/mvcc/`, following the precedent of EVM-compatible chains (Avalanche C-Chain, Moonbeam, Hedera) that pair EVM compatibility with non-Ethereum consensus.

Each account maintains a **dual cryptographic identity**: ECDSA over secp256k1 for Ethereum wallet compatibility and Ed25519 for native operations, following Hedera's production-proven multi-scheme pattern. Address handling accepts both embedded 20-byte EVM addresses and full 32-byte public keys, hashing the latter with Keccak-256 (`core/execution/src/types.rs`).

0.4.2 3.2 AI Precompiled Contracts

[Implemented] The LVM extends the EVM with several families of AI-specific precompiles (`core/execution/src/` Precompiles execute native code rather than EVM bytecode, enabling operations prohibitively expensive as Solidity, following Ethereum's own precompile pattern (ecrecover at 0x01, and so on, which Citrate also provides at 0x01–0x09). The February 2026 draft anticipated five precompiles at 0x1000–0x1004; the implemented address map is richer and is documented here as it actually exists.

Table 1. Implemented AI precompile families.

Address range	Family	Representative operations	Status
0x0100–0x0106	Inference (`inference.rs`)	deploy, inference, batch, metadata, proof-verify, benchmark, encryption; 0x0101/0x0102 attestation-gated	Implemented
0x0107–0x0109	Verification (`verify.rs`)	0x0107 Poseidon TENSOR_COMMIT, 0x0108 Halo2-KZG INFERENCE_PROOF_VERIFY, 0x0109 MERKLE_VERIFY_TENSOR	Implemented (0x0108 feature-gated)
0x010A–0x010F	Deterministic Q16.16 compute (`compute.rs`)	MATMUL, DOT, SOFTMAX, RELU, LINEAR, TRANSPOSE	Implemented
0x0110–0x0111	Learning (`q16/belnap.rs`, `q16/routing.rs`)	Belnap 4-valued aggregation, routing	Implemented
0x0130	Recursive-fold CommD verifier (`commd_fold_verify.rs`)	proof-of-storage fold verification	Implemented

Address range	Family	Representative operations	Status
0x0200–0x0209	x402 payments (`x402.rs`)	authorization verify, EIP-712 verify	Implemented

Gas costs are calibrated to computational cost and include reduced-circuit versions for the proof-verifying precompiles (`verify.rs:89-111`).

0.4.3 3.3 Verifiable Inference

[Implemented: signature and optimistic tiers; Feature-gated: ZK tier.] Inference results carry a cost-security tradeoff across three tiers. Signature-based verification relies on the provider's staked identity, suitable for low-value queries where stake exceeds the gain from cheating. Optimistic verification posts a result with a fraud-proof challenge window; a challenger who re-executes and proves fraud triggers slashing (Section 5; `NematocystSlashing.sol`). ZK-SNARK verification provides cryptographic rather than economic security: the **Halo2-KZG verifier is real code** (`core/execution/src/zkp/halo2/mod.rs:199`, using `verify_proof_multi`, `KZGCommitmentScheme`, `VerifierSHPLONK`), invoked by precompile 0x0108 (`verify.rs:118`), but it is **compiled behind the `halo2-substrate` cargo feature**; the default build returns a discoverable "feature absent" error (`mod.rs:132`). We state this plainly: proving and verifying are implemented and tested (`tests/inference_proof_verify_real_srs.rs`, `tests/porep_proof_verify_e2e.rs`), but off by default. Paper X, *The Substrate of Verifiable Inference*, specifies this layer in full.

0.4.4 3.4 Model Registry and Deterministic Compute

[Implemented] Model registration is realized through the inference precompile's deploy path (0x0100) together with on-chain contracts; each registered model carries a deterministic weight hash, an IPFS CID for decentralized weight storage, architecture metadata, version history, and the registrant's staked identity. Deterministic **Q16.16 fixed-point compute** (`core/execution/src/precompiles/q16/o`) backs both the tensor primitives (0x010A–0x010F) and the learning precompiles, with cross-platform determinism fixtures (`tests/cross_platform/q16_determinism.rs`) so that inference used inside consensus is bit-reproducible across nodes.

0.5 4. Orchestration Layer: Model Context Protocol

[Implemented] The MCP layer (`core/mcp/`) exposes OpenAI- and Anthropic-compatible REST endpoints, `/v1/models`, `/v1/chat/completions`, `/v1/embeddings`, `/v1/messages`, and `/v1/jobs` for async work, alongside the JSON-RPC `eth_*` surface at port 8545 and custom `citrate_*` DAG queries. The February 2026 draft listed MCP as *Specified*; the compatible REST surface is now implemented, letting contracts and off-chain clients discover models and route inference through a standard interface. A full decentralized routing marketplace, in which hosts set inference prices and consumers select providers by capability, load, and cost, remains **[Specified]** and depends on the marketplace contracts.

0.6 5. SALT Tokenomics

[Implemented in contracts; economic policy Specified] SALT is Citrate's native token, fixed supply 1 billion, functioning as gas, staking collateral, and governance weight. The economic policy

below is design; the mechanisms enforcing it are deployed contracts on chain 40204.

Table 2. SALT distribution (design).

Allocation	Percentage	Amount	Vesting
Mining rewards	50%	500,000,000	emitted over network lifetime via block rewards
Ecosystem development	25%	250,000,000	grants, partnerships, developer incentives
Treasury	10%	100,000,000	DAO-governed reserve
Team	15%	150,000,000	4-year vesting, 1-year cliff

Validators that host registered models and serve verifiable inference earn bonuses beyond the base block reward, aligning validation with AI-infrastructure provision. The enforcing contracts are live: **market-maker allocation** (`MarketMakerAllocation.sol`, `0xa87fae5c…`), **contribution accounting** (`ContributionAccounting.sol`, `0xcdd24773…`, Paper VII), **treasury governance** (`TreasuryGovernor.sol`, `0x62e268f2…`, Paper VIII), and **tiered Byzantine slashing** (`NematocystSlashing.sol`, `0xfeb23abd…`, Paper IX), which implements stake, unstake, slash, an `isSlashable` gate, and a correlated-failure multiplier. Validator membership and staking are enforced by `CitrateMemberSBT.sol` (`0xAD826D04…`) and `MembershipStakeVault.sol` (`0x04c32967…`). In keeping with Citrate Inc.'s public-communication policy, this paper makes no claim of day-one cash earnings from compute; SALT's availability on a licensed exchange is pending.

0.7 6. Security Analysis

0.7.1 6.1 Consensus Security

The GhostDAG + BFT consensus inherits well-studied properties. Safety (no conflicting finalized states) holds under the Byzantine assumption $f < n/3$ [11]; with a 100-validator committee requiring 67 signatures, this tolerates up to 33 Byzantine validators. Liveness holds under GhostDAG's properties when fewer than half of validators are Byzantine [1, 2]. The PoW component provides Sybil resistance for block production while the BFT finality layer provides fast deterministic finality, avoiding the finality delays of pure PoW while retaining permissionless participation.

0.7.2 6.2 Inference Verification Security

The three-tier verification system is defense in depth. The signature tier is bounded by the provider's stake, an explicit and auditable economic bound. The optimistic tier depends on an honest challenger being online, mitigated by awarding a share of the slashed stake to the challenger. The ZK tier, when its feature is enabled, provides cryptographic rather than economic security at higher gas cost. **Open problem:** ZK proof generation for large-model inference remains expensive; the practical envelope today is small models and high-value queries. This is why 0x0108 ships feature-gated rather than always-on, and why Paper X treats TEE attestation (`TEEAttestationRegistry.sol`, `0x4df26aae…`) as a complementary trust path.

0.7.3 6.3 Dual Cryptography

[**Implemented**] Each account maintains ECDSA (secp256k1) and Ed25519 key pairs. Ed25519 gives faster verification and deterministic signatures, eliminating the k-nonce vulnerability that has caused ECDSA key exposure in production systems. Hedera has operated a similar dual-cryptography design in production since 2019, demonstrating viability at scale.

0.8 7. Comparative Position

Citrate sits at the intersection of DAG-based consensus, EVM-compatible execution, and decentralized AI. We compare along the dimension most relevant to each neighbor rather than forcing one table across incommensurable systems.

7.1 DAG consensus: Kaspa. Kaspa is the production reference for GhostDAG. After Crescendo (May 2025) it runs at 10 BPS with k=124 and max parents 16, reaching a record ~3,585 TPS [30]. Citrate runs more conservatively (2-second blocks, k=18, max parents 10) to carry AI-augmented block overhead, and adds a general-purpose execution layer with AI precompiles atop the same consensus family, which Kaspa does not provide.

7.2 EVM chains and L2s. Against Arbitrum, Optimism, zkSync, Avalanche C-Chain, Moonbeam, and Hedera, Citrate's differentiator is the AI precompile suite: no existing EVM-compatible chain provides native precompiles for model registration, verifiable inference, deterministic tensor compute, and paraconsistent aggregation. Applications needing on-chain AI otherwise implement it as expensive Solidity or trust off-chain oracles.

7.3 Decentralized AI: Bittensor and the AI-native L1 category. Bittensor is an AI coordination marketplace scoring miner outputs via Yuma consensus; Citrate is a general blockchain with composable AI operations. Comparing them on TPS is a category error. The meaningful comparison is integration approach: Bittensor isolates AI tasks in subnets; Citrate embeds AI operations in the execution layer, composable with arbitrary contract logic. Newer entrants sharpen the contrast. Proof-of-logits designs (Ambient) make inference itself the mining function; optimistic and TEE-attested inference rollups [32] verify generative inference off the hot path; the DeAI systematization [31] maps the category. Citrate's position is that verifiable inference should be a precompile a DeFi risk engine or a model-governed DAO can call inline, not a separate network to bridge to.

0.9 8. Open Problems and Future Work

8.1 Federated learning at scale. The learning path of Paper II is implemented at the execution and daemon layers (Belnap precompile 0x0110; `learning-daemon/src/aggregator.rs`; `LearningCheckpoint` at `learning/src/checkpoint.rs:14`). Open questions remain on the minimum viable meta-model at checkpoint-interval timescales, how LoRA-adapter composition scales with accumulated adapters, and provable convergence of the recursive learning loop under Byzantine conditions.

8.2 Throughput scaling. Kaspa's trajectory from 1 to 10 BPS suggests GhostDAG systems can raise block rates substantially. Citrate's 2-second cadence is deliberately conservative for AI-augmented blocks; with better embedding compression the rate can rise via governance, proportionally increasing throughput. This is an engineering optimization, not an architectural limit.

8.3 Hardware optimization. Paper V (ATIS) proposes analog middleware for energy-efficient attention pruning on FPAA hardware. If realized, it would optimize the deterministic tensor precompiles; it remains a research proposal with no prototype built.

8.4 Cross-chain value transfer. Paper VI describes the Memetic Money Portal. The honest current state is that Citrate implements a `WrappedSALT` EIP-3009 gasless-transfer path (`0xaa918302…`) plus market-maker allocation and a SALT/USD feed embedded in `ComputePricingOracle.sol` (`0xdcebd5ec…`); there is no SNAP/SALT lock-mint bridge contract, and Paper VI is being revised to describe the mechanism as it exists rather than as a bridge.

0.10 9. Conclusion

Citrate is a BlockDAG network that adds AI-native execution to GhostDAG consensus. The implemented system provides parallel block production with dual (depth + committee-BFT) finality, EVM compatibility via REVM, families of AI precompiles for inference, verifiable proof checking, deterministic tensor compute and paraconsistent aggregation, dual cryptographic identity, an MCP-compatible REST surface, and SALT tokenomics enforced by 39 deployed contracts on chain 40204.

The distinguishing contribution is not any single component but their combination: a general-purpose programmable blockchain with native, composable AI operations on a consensus architecture that carries federated learning as a first-class path. Whether that combination produces capabilities beyond its parts is an empirical question the companion papers address theoretically (II–III), from a hardware perspective (V), economically (VI–VII), through governance (VIII), and through the biological design lens (IX); Paper X specifies the verifiable-inference substrate and Paper XI presents the Neuroarchitectural Transformer that runs on it. We have kept the earlier draft's discipline of separating implemented from specified and labelling targets as targets, on the conviction that credibility, not headline numbers, is what builds lasting systems.

0.11 Acknowledgments

Drafting and literature triage were assisted by AI systems; all mechanical claims were verified by the authors against the referenced source files and deployed contract addresses on chain 40204. This work received no external funding.

0.12 References

[1] Sompolinsky, Y., & Zohar, A. (2015). Secure high-rate transaction processing in Bitcoin. *Financial Cryptography and Data Security*, 507–527. [2] Sompolinsky, Y., & Zohar, A. (2018). PHANTOM and GHOSTDAG: A scalable generalization of Nakamoto consensus. *IACR Cryptology ePrint Archive.* [3] Keidar, I., Kokoris-Kogias, E., Naor, O., & Spiegelman, A. (2021). All you need is DAG. *Proc. ACM PODC*, 165–175. [4] Danezis, G., Kokoris-Kogias, L., Sonnino, A., & Spiegelman, A. (2022). Narwhal and Tusk. *Proc. EuroSys.* [5] Spiegelman, A., Giridharan, N., Sonnino, A., & Kokoris-Kogias, L. (2022). Bullshark: DAG BFT protocols made practical. *Proc. CCS.* [6] McMahan, B., et al. (2017). Communication-efficient learning of deep networks from decentralized data. *AISTATS*, 1273–1282. [7] Shazeer, N., et al. (2017). Outrageously large neural networks: the sparsely-gated mixture-of-experts layer. *ICLR.* [8] Hu, E. J., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. *ICLR 2022.* [9] Vaswani, A., et al. (2017). Attention is all you need. *NeurIPS 30.* [10] Buterin, V. (2014). Ethereum: a next-generation smart contract and decentralized application platform. [11] Castro, M., & Liskov, B. (1999). Practical Byzantine fault tolerance. *OSDI*, 173–186. [12] Ben-Sasson, E., et al. (2014). Succinct non-interactive zero knowledge for a von Neumann architecture. *USENIX Security*, 781–796. [13] Rao, J., & Opentensor Foundation. (2021). Bittensor: a peer-to-peer intelligence market. [14] Belnap, N. D. (1977). A useful four-valued logic. In Dunn, J. M., Epstein, G. (eds), *Modern Uses of Multiple-Valued Logic*, Episteme vol. 2, Reidel, Dordrecht. [15] Kirkpatrick, J., et al. (2017). Overcoming catastrophic

forgetting in neural networks. *PNAS*, 114(13), 3521–3526. [16] Nakamoto, S. (2008). Bitcoin: a peer-to-peer electronic cash system. [17] Ilharco, G., et al. (2023). Editing models with task arithmetic. *ICLR.* [18] Yadav, P., et al. (2023). TIES-Merging: resolving interference when merging models. *NeurIPS.* [19] Biderman, D., et al. (2024). LoRA learns less and forgets less. *Transactions on Machine Learning Research.* (First author Dan Biderman; corrected from the February 2026 draft, which listed "Biderman, S.") [20] Weissbourd, B., et al. (2021). A genetically tractable jellyfish model for systems and evolutionary neuroscience. *Cell*, 184(24), 5854–5868. [21] Pallasdies, F., et al. (2019). From single neurons to behavior in the jellyfish *Aurelia aurita*. *eLife*, 8, e50084. [22] Klosowski, L. (2023). Mentor/Mentee Relativity: Organizational Learning in Mentorship-Driven Swarms. Unpublished internal note, Citrate Inc. [23] Anthropic. (2024). Model Context Protocol Specification. Technical report. [27] Yin, M., et al. (2019). HotStuff: BFT consensus with linearity and responsiveness. *PODC*, 347–356. [28] Priest, G. (2006). *In Contradiction: A Study of the Transconsistent.* Oxford University Press. [30] Kaspa Network. (2025). KIP-14: Crescendo Hardfork. Activated May 5, 2025. [31] Zhang, W., et al. (2024). SoK: Blockchain-Based Decentralized AI. arXiv:2411.17461. [32] Authors TBD. (2025). Optimistic TEE-Rollups: A Hybrid Architecture for Scalable and Verifiable Generative AI Inference on Blockchain. arXiv:2512.20176. *(Verify author list before final submission.)*

0.13 Appendix A: Protocol Parameters

Table A1. Citrate protocol configuration (reconciled against code, Aug 2026).

Parameter	Value	Source
Chain ID	40204 (testnet)	`testnet-config.toml:2`; `genesis_model.rs:269`
Block time	~2 s (testnet), 1 s (devnet)	`testnet-config.toml:4`; `block_builder.rs:76`
Max parents	10 (1 selected + 9 merge)	`core/consensus/src/types.rs:179`
k parameter	18	`core/consensus/src/types.rs:178`
Checkpoint interval	50 blocks	`core/consensus/src/checkpoint.rs:101`
Committee size	100 validators	`checkpoint.rs`
BFT quorum	67 (2/3+1)	`checkpoint.rs:88,103`
Vote domain separator	`CITRATE-CHECKPOINT-V1`	`checkpoint.rs:59`
Signature schemes	ECDSA (secp256k1) + Ed25519	`core/execution/src/types.rs`
Deployed contracts (40204)	39	`contracts/DEPLOYED_ADDRESSES.md`
Core test count	3,517	`cargo test --workspace` (grep)
Parallel-execution gate	8-worker 2× 1-worker	`execution/benches/tps_parallel.rs:10`
TPS figure (target, not benched)	5,000 sustained / 10,000 ceiling	`README.md:65`
SALT total supply	1,000,000,000	design

This paper is part of the Gradient Papers series, published by Citrate Inc. Correspondence: Larry@citrate.ai